THERMO PROPERTY API

Integration Guide

Stream properties, enthalpy and heat duty, and flow versus valve opening

Version	3.0.0
Date	23 October 2026
Base URL	http://192.168.11.216:4300/api
Format	HTTP and JSON (UTF-8)
Access	Company network or VPN
Authentication	None in the current deployment

Contents

Section	Title	Endpoint
1	Overview	
2	Request and response format	
3	Errors	
4	Terms used in this guide	
5	CAS numbers	
6	Stream properties	POST /eos_stream_calc
7	Enthalpy and heat duty	POST /eos_enthalpy_calc
8	Flow versus valve opening	POST /vent_flow_calc
9	Health check	GET /health

1. Overview

The Thermo Property API calculates thermodynamic results for process streams. This guide describes the three calculations you will use and a health check. All of them are called the same way, and all answer in the same format.

API	Section	The question it answers	You send	You get
Stream properties	6	Which phases does the stream have at this temperature and pressure, and what are their amounts and properties?	T, P, components (CAS), mole fractions	Phase situation; for each phase its fraction, molecular weight, heat capacity and density; values of the mixture
Enthalpy and heat duty	7	How much heat is needed to take the stream from an inlet to an outlet condition?	Inlet T and P, outlet T and P, components (CAS), mole fractions	Enthalpy at both conditions and the heat duty (kJ/mol and kJ/kg)
Flow versus valve opening	8	What flow does the valve give at a given opening?	Your measured opening and flow points, a degree, the openings to estimate	The fitted equation with its statistics, and the estimated flow at each opening

Four rules for every request

- **Components are CAS numbers**, for example `7732-18-5` for water. Names are not accepted: a name returns an error that tells you which entry to change (section 5).
- **Compositions are mole fractions, not mole percent.** 25 % is written 0.25, and all fractions of a stream add up to 1.
- **Units:** temperature in K and pressure in Pa. In the Vent API you choose the unit of the flow, and the answer comes back in that same unit.
- Send **POST** with a JSON body and the header `Content-Type: application/json`. Every answer has the same layout (section 2), and every error has a code (section 3).

Address

Base URL	http://192.168.11.216:4300/api
Example	http://192.168.11.216:4300/api/eos_stream_calc
Time limit	A calculation that takes longer than 30 seconds is stopped (error `calculation_timeout`).

2. Request and response format

Request

Send the fields listed in each API section as one JSON object. The two fields that describe a stream are the same in sections 6 and 7:

Field	Type	Rule
`components`	array of text	**CAS numbers only**, for example `"7732-18-5"` (water). Names such as `"water"` are rejected with an error. At least one, no repeats.
`zs`	array of numbers	**Mole fractions, not mole percent**: 0.25 means 25 %. Same order and length as `components`, none negative, adding up to 1. A small difference from 1 is rescaled (`meta.zs_normalized` is then true). A component with 0 is allowed and is left out of the calculation.

Example of a correct stream: water 25 % and hexane 75 % is written like this.

```
"components": ["7732-18-5", "110-54-3"],
"zs": [0.25, 0.75]
```

Successful response (HTTP 200)

Every success has the same four parts. The complete response of each API is shown in its own section.

Part	Meaning
`status`	Always `"success"`.
`data`	The results. Their fields are explained in each API section.
`units`	The unit of each value in `data`.
`meta`	`request_id` (quote it when you contact support), `elapsed_ms` (server time in milliseconds) and, for requests with a composition, `zs_normalized` (true if your fractions were rescaled to add up to 1).

A phase or value that does not exist is returned as `null`, never as 0, and this is not an error. Numbers are returned at full precision; round them in your own software.

Error response (HTTP 4xx or 5xx)

```
{
  "status": "error",
  "error": {
    "code": "validation_error",
    "message": "Request body is invalid.",
    "details": [ { "field": "T", "message": "Input should be greater than 0" } ]
  },
  "request_id": "a1b2c3d4e5f6"
}
```

Use `error.code` in your software; it never changes. `message` is for people, and `details` gives the specifics (for `validation_error` a list of fields and what is wrong with each).

3. Errors

Always check the HTTP status first: 200 is success, anything else is an error in the format of section 2.

Error code	HTTP	Cause	What to do
`validation_error`	422	The request is invalid: bad JSON, wrong type, a component that is not a CAS number, T or P not above 0, lists of different length, a negative or repeated entry, too few data points.	Read `error.details` and fix the listed fields.
`unknown_component`	422	A CAS number is written correctly but is not in the component database.	Check the number; use another component.
`property_unavailable`	422	Data the model needs for a component is missing.	Remove or replace that component.
`flash_failed`	422	The equilibrium calculation failed for this state.	Check temperature, pressure and composition.
`fit_failed`	422	Vent API only: too few different opening values for the degree.	Send more different openings, or lower the degree.
`unauthorized`	401	An API key is enabled and the `X-API-Key` header is missing or wrong.	Send the correct key.
`not_found`	404	The URL does not exist.	Check the path, including `/api`.
`method_not_allowed`	405	Wrong HTTP method, for example GET on a calculation.	Use POST.
`calculation_failed`	500	The model could not be built.	Report it with the `request_id`.
`internal_error`	500	Unexpected server error.	Report it with the `request_id`.
`calculation_timeout`	504	The calculation took longer than 30 seconds.	Simplify the input, or contact support.

Example: a name instead of a CAS number (HTTP 422)

```
{
  "status": "error",
  "error": {
    "code": "validation_error",
    "message": "Request body is invalid.",
    "details": [
      {
        "field": "components",
        "message": "components must be CAS numbers such as 7732-18-5 (water); names are not accepted. Not
accepted: 'water' (a name, not a CAS number), 'hexane' (a name, not a CAS number)"
      }
    ]
  },
  "request_id": "a1b2c3d4e5f6"
}
```

Handling errors in your software

- Decide by HTTP status and `error.code`, never by the text of `message`.
- Do not repeat a 422 request unchanged: the input has to change. A 504 may be repeated once with simpler input.
- Log the `request_id` of every failure; support can find the request with it.

4. Terms used in this guide

Term	Meaning
CAS number	The international identifier of a chemical, for example 7732-18-5 for water. The API accepts components only as CAS numbers (section 5).
Mole fraction	The moles of one component divided by the total moles of the stream. It is a number from 0 to 1 (0.25 = 25 %); all mole fractions of a stream add up to 1. It is never given in percent.
Phase	A physically separate part of the stream: **vapor** (gas), **aqueous** liquid (the water-rich liquid) and **organic** liquid (the hydrocarbon-rich liquid).
Vapor fraction, V	The share of the stream's moles that is vapor: 0 means all liquid, 1 means all vapor. The liquid fraction is L = 1 − V.
Phase fraction, beta	The share of the stream's moles that is in one particular phase. The betas of the phases that exist add up to 1.
case	Which phases the calculation found: `vapor-only`; `liquid-only`; `two-phase vapor-liquid (VL)` = vapor and one liquid; `two-phase liquid-liquid (LL)` = aqueous and organic liquid, no vapor; `three-phase vapor-liquid-liquid (VLLE)` = vapor plus both liquids.
Peng-Robinson equation of state	The thermodynamic model behind sections 6 and 7. It describes real gas and liquid behaviour and lets the calculation decide which phases exist.
MW	Molecular weight: the mass of one mole, in g/mol.
Cp	Molar heat capacity: the energy needed to warm one mole by one kelvin, in J/mol/K. `Cp_mass_kJ_kgK` is the same per kilogram, in kJ/kg/K.
rho_mass, rho_molar	Density: mass per volume in kg/m³, and moles per volume in kmol/m³.
Enthalpy	The energy content of the stream, per mole (kJ/mol) or per kilogram (kJ/kg), measured from the model's reference state. Only differences between two conditions are meaningful.
Heat duty	Outlet enthalpy minus inlet enthalpy: the heat needed to take the stream from inlet to outlet. Positive means heat must be added (heating, vaporisation); negative means heat is released (cooling, condensation). `heat_duty_molar` is per mole (kJ/mol), `sp_duty` per kilogram (kJ/kg).
Interaction parameter (kij)	A number the model uses for how strongly two components affect each other. For ethylene and ethane a calibrated value of 0.015 is used; `ethylene_ethane_kij_override` tells you when.
Opening	The valve opening in % (Vent API).
Polynomial fit, degree	A curve Flow = a·Opening^d + … + c·Opening + e through your points. The degree d is the highest power: 1 is a straight line, 2 a parabola.
LINEST	The Excel function for the same least-squares fit. The coefficients and statistics of the Vent API are those LINEST gives.
R^2	How well the curve follows your points: 1 is a perfect fit, values near 0 are a poor one.
Standard error of the estimate (SE y)	The typical distance between your points and the curve, in the unit of your flow.
F statistic, residual degrees of freedom	Standard statistics of the fit. The degrees of freedom are the number of points minus the number of coefficients. The F statistic is `null` when it has no meaning.
Extrapolated	An opening outside the range of your data. The estimate is less reliable there, so it is marked, and by default evaluated at the nearest end of your range.
null	The value does not exist or is not defined, for example the vapor of a stream that has no vapor.
request_id	A short code identifying one request. Quote it when you contact support.

5. CAS numbers

A CAS number (CAS Registry Number) is the worldwide identifier of a chemical. It is printed on safety data sheets and in every chemical database, and it does not depend on language or spelling. The API identifies components **only** by CAS number.

How to write it

- Three groups of digits joined by hyphens: 2 to 7 digits, then 2 digits, then 1 digit. Examples: `7732-18-5` (water), `74-82-8` (methane), `74-84-0` (ethane), `74-85-1` (ethylene), `110-54-3` (n-hexane).
- The last digit is a check digit, so a typing error is detected. `7732-18-4` is not accepted.
- Spaces around the number are ignored. Names, formulas and other spellings (`water`, `H2O`) are not accepted.
- Each CAS number may appear only once in a request.

What the API answers

You send	The API answers
A name, for example `"water"`	`validation_error`: the message names every entry that is not a CAS number (example in section 3).
A mistyped number, for example `7732-18-4`	`validation_error`: invalid CAS check digit (below).
A correct number the database does not contain	`unknown_component`: the message and `error.details.component` name the number (below).
The same CAS number twice	`validation_error`: duplicate components are not allowed.

A mistyped number (HTTP 422):

```
{
  "status": "error",
  "error": {
    "code": "validation_error",
    "message": "Request body is invalid.",
    "details": [
      {
        "field": "components",
        "message": "components must be CAS numbers such as 7732-18-5 (water); names are not accepted. Not
accepted: '7732-18-4' (invalid CAS check digit (typing error?))"
      }
    ]
  },
  "request_id": "a1b2c3d4e5f6"
}
```

A correct number that is not in the component database (HTTP 422):

```
{
  "status": "error",
  "error": {
    "code": "unknown_component",
    "message": "CAS number '9999-99-9' was not found in the component database.",
    "details": { "component": "9999-99-9" }
  },
  "request_id": "a1b2c3d4e5f6"
}
```

6. Stream properties

POST /eos_stream_calc

http://192.168.11.216:4300/api/eos_stream_calc

Tells you which phases a stream has at a given temperature and pressure, and for every phase that exists its amount, molecular weight, heat capacity and density, plus the values of the whole mixture. A Peng-Robinson equation of state with a three-phase calculation (vapor, aqueous liquid, organic liquid) is used, so water and hydrocarbons are handled correctly. Water is optional.

Request

Field	Type	Rule
`T`	number	Temperature in kelvin (K), above 0.
`P`	number	Pressure in pascal (Pa), above 0. 1 atm = 101325 Pa.
`components`	array of text	**CAS numbers only**, for example `"7732-18-5"` (water). Names such as `"water"` are rejected with an error. At least one, no repeats.
`zs`	array of numbers	**Mole fractions, not mole percent**: 0.25 means 25 %. Same order and length as `components`, none negative, adding up to 1. A small difference from 1 is rescaled (`meta.zs_normalized` is then true). A component with 0 is allowed and is left out of the calculation.

```
{ "T": 300, "P": 101325, "components": ["7732-18-5", "110-54-3"], "zs": [0.5, 0.5] }
```

Example: water (7732-18-5) and hexane (110-54-3), 0.5 mole fraction each, at 300 K and 101325 Pa.

Response

Field	Type	Unit	Meaning
`case`	text	–	Which phases were found: `vapor-only`, `liquid-only (single liquid phase)`, `two-phase vapor-liquid (VL)`, `two-phase liquid-liquid (LL, aqueous+organic)` or `three-phase vapor-liquid-liquid (VLLE)`.
`V`, `L`	number	mole fraction	Vapor fraction, and total liquid fraction (aqueous plus organic). V + L = 1.
`n_phases`	integer	count	Number of phases found.
`n_components_used`	integer	count	Components that took part (mole fraction above 0).
`vapor`, `aqueous`, `organic`	object or null	–	One object per phase, or null if that phase does not exist. It holds `beta` (share of the total moles in this phase), `MW` (g/mol), `Cp` (J/mol/K), `rho_mass` (kg/m³) and `rho_molar_kmol_m3` (kmol/m³).
`mixture`	object	–	Values of the whole stream: `MW` (g/mol), `Cp` (J/mol/K), `Cp_mass_kJ_kgK` (kJ/kg/K), `rho_mass` (kg/m³), `rho_molar_kmol_m3` (kmol/m³).

Complete response for the example above (HTTP 200):

```
{
  "status": "success",
  "data": {
    "case": "two-phase liquid-liquid (LL, aqueous+organic)",
    "V": 0.0,
    "L": 1.0,
    "n_phases": 2,
    "n_components_used": 2,
    "vapor": null,
    "aqueous": {
      "beta": 0.4904076,
      "MW": 18.01528,
      "Cp": 81.16374,
      "rho_mass": 847.61,
      "rho_molar_kmol_m3": 47.0495
    },
    "organic": {
      "beta": 0.5095924,
      "MW": 84.89234,
      "Cp": 186.3908,
      "rho_mass": 663.8171,
      "rho_molar_kmol_m3": 7.819517
    },
    "mixture": {
      "MW": 52.09532,
      "Cp": 134.7867,
      "Cp_mass_kJ_kgK": 2.587308,
      "rho_mass": 689.1598,
      "rho_molar_kmol_m3": 13.22882
    }
  },
  "units": {
    "V": "mole fraction",
    "L": "mole fraction",
    "n_phases": "count",
    "n_components_used": "count",
    "beta": "mole fraction of the total mixture (per phase)",
    "MW": "g/mol",
    "Cp": "J/mol/K",
    "Cp_mass_kJ_kgK": "kJ/kg/K",
    "rho_mass": "kg/m^3",
    "rho_molar_kmol_m3": "kmol/m^3"
  },
  "meta": { "request_id": "a1b2c3d4e5f6", "elapsed_ms": 2.4, "zs_normalized": false }
}
```


Reading the example

- `case` shows two liquid phases and no vapor, so `V` is 0 and `vapor` is `null`.
- The water-rich liquid is `aqueous` (`MW` 18.015) and the hexane-rich liquid is `organic`. Their `beta` values (0.4904 and 0.5096) add up to 1.

Errors

Error code	HTTP	When it occurs here
`validation_error`	422	A field is missing, has the wrong type or breaks a rule: a component that is not a CAS number (names are not accepted), T or P not above 0, lists of different length, a negative or repeated entry. `error.details` lists each field.
`unknown_component`	422	A CAS number is correctly written but is not in the component database. `error.details.component` names it.
`property_unavailable`	422	Data the model needs for a component is missing. `error.details.components` lists them.
`flash_failed`	422	The equilibrium calculation did not work for this state. Check the conditions and the composition.
`calculation_timeout`	504	The calculation took longer than 30 seconds.

Notes

- The liquid with the most water is reported as **aqueous**; the other liquid is **organic**. With a single liquid, it is aqueous if it is more than half water.
- The first request for a new set of components builds the model and can take a few seconds (about 3 s for 25 components). Later requests for the same components take milliseconds.

7. Enthalpy and heat duty

POST /eos_enthalpy_calc

http://192.168.11.216:4300/api/eos_enthalpy_calc

Gives the enthalpy of one stream at an inlet condition and at an outlet condition, and the heat duty between them: the heat that has to be added (or removed) to take the stream from the inlet to the outlet. The composition is the same at both conditions. A change of phase between inlet and outlet, such as vaporisation, is included. The calculation uses the same Peng-Robinson model as section 6.

Request

Field	Type	Rule
`T_in`, `P_in`	number	Inlet temperature in K and pressure in Pa, both above 0.
`T_out`, `P_out`	number	Outlet temperature in K and pressure in Pa, both above 0.
`components`	array of text	**CAS numbers only**, for example `"7732-18-5"` (water). Names such as `"water"` are rejected with an error. At least one, no repeats.
`zs`	array of numbers	**Mole fractions, not mole percent**: 0.25 means 25 %. Same order and length as `components`, none negative, adding up to 1. A small difference from 1 is rescaled (`meta.zs_normalized` is then true). A component with 0 is allowed and is left out of the calculation.

```
{
  "T_in": 300,
  "P_in": 254920,
  "T_out": 368,
  "P_out": 254720,
  "components": [
    "74-82-8", "74-86-2", "74-85-1", "74-84-0", "74-99-7", "115-07-1", "74-98-6", "106-99-0",
    "106-98-9", "106-97-8"
  ],
  "zs": [0.0002, 0.0043, 0.606, 0.3794, 0.0001, 0.0072, 0.0014, 0.0005, 0.0005, 0.0004]
}
```

Example: a C2 stream of 10 components (CAS numbers, mole fractions adding up to 1) heated from 300 K to 368 K.

Response

Field	Type	Unit	Meaning
`inlet`, `outlet`	object	–	The stream at that condition: `vapor_fraction` (mole fraction), `molecular_weight` (g/mol), `enthalpy_molar` (kJ/mol), `enthalpy_mass` (kJ/kg).
`duty.heat_duty_molar`	number	kJ/mol	Outlet enthalpy minus inlet enthalpy, per mole of stream. Positive: heat is absorbed. Negative: heat is released.
`duty.sp_duty`	number	kJ/kg	The same duty per kilogram of stream (specific duty).
`n_components_used`	integer	count	Components that took part (mole fraction above 0).
`ethylene_ethane_kij_override`	boolean	–	true when ethylene and ethane are both in the stream and the calibrated interaction parameter 0.015 was used for that pair.

Complete response for the example above (HTTP 200):

```
{
  "status": "success",
  "data": {
    "inlet": {
      "vapor_fraction": 1.0,
      "molecular_weight": 28.97064,
      "enthalpy_molar": -0.04241087,
      "enthalpy_mass": -1.463926
    },
    "outlet": {
      "vapor_fraction": 1.0,
      "molecular_weight": 28.97064,
      "enthalpy_molar": 3.439142,
      "enthalpy_mass": 118.7113
    },
    "duty": { "heat_duty_molar": 3.481552, "sp_duty": 120.1752 },
    "n_components_used": 10,
    "ethylene_ethane_kij_override": true
  },
  "units": {
    "vapor_fraction": "mole fraction",
    "molecular_weight": "g/mol",
    "enthalpy_molar": "kJ/mol",
    "enthalpy_mass": "kJ/kg",
    "heat_duty_molar": "kJ/mol",
    "sp_duty": "kJ/kg",
    "n_components_used": "count"
  },
  "meta": { "request_id": "a1b2c3d4e5f6", "elapsed_ms": 36.0, "zs_normalized": false }
}
```


Reading the example

- The stream stays a vapor (`vapor_fraction` 1) from 300 K to 368 K, so the duty is the heat needed to warm it.
- `heat_duty_molar` 3.481552 kJ/mol is the same energy as `sp_duty` 120.1752 kJ/kg: 28.97064 g/mol times 120.1752 kJ/kg gives 3.481552 kJ/mol.
- `ethylene_ethane_kij_override` is true because both ethylene (74-85-1) and ethane (74-84-0) are present.

Errors

Error code	HTTP	When it occurs here
`validation_error`	422	A field is missing, has the wrong type or breaks a rule: a component that is not a CAS number (names are not accepted), T or P not above 0, lists of different length, a negative or repeated entry. `error.details` lists each field.
`unknown_component`	422	A CAS number is correctly written but is not in the component database. `error.details.component` names it.
`property_unavailable`	422	Data the model needs for a component is missing. `error.details.components` lists them.
`flash_failed`	422	The equilibrium calculation did not work at the inlet or at the outlet condition; the message says which.
`calculation_timeout`	504	The calculation took longer than 30 seconds.

Notes

- Heating water (7732-18-5) from 300 K to 400 K at 101325 Pa gives `sp_duty` 2717.545 kJ/kg, because the liquid becomes vapor and the latent heat is included.
- Swapping inlet and outlet gives the same duty with the opposite sign (cooling releases heat).
- Enthalpy values refer to the model's own reference state, so use differences such as the duty, not the absolute numbers.
- The first request for a new set of components builds the model (about 3 s); later requests take milliseconds.

8. Flow versus valve opening

POST /vent_flow_calc

http://192.168.11.216:4300/api/vent_flow_calc

Takes your measured points of flow versus valve opening, fits a polynomial through them, and estimates the flow at the openings you ask for. The fit is the same as Excel's LINEST function with Flow as y and Opening as x, and the response gives the equation and the usual fit statistics. No components, temperature or pressure are involved.

Request

Field	Type	Rule
`opening_values`	array of numbers	Valve opening of each data point, in %. At least 3 points.
`flow_values`	array of numbers	Flow measured at each opening, in the unit of your data (for example kg/h). Same length as `opening_values`.
`degree`	integer	Polynomial degree you ask for, 1 to 10 (1 = straight line, 2 = parabola). If there are too few points it is reduced automatically to the points minus 2.
`estimate_at`	array of numbers	Openings (%) at which to estimate the flow. May be empty.
`clip_to_fit_range`	true or false	Default true: an opening outside the range of your data is evaluated at the nearest end of that range (it is still marked as extrapolated).
`min_flow`	number	Default 0. Estimated flows below this value are raised to it.

```
{
  "opening_values": [19.7, 49.6, 89.7],
  "flow_values": [48601, 122050, 220698],
  "degree": 3,
  "estimate_at": [5, 50, 110],
  "clip_to_fit_range": true,
  "min_flow": 0.0
}
```

Example: three data points, degree 3 requested, flow estimated at 5 %, 50 % and 110 % opening.

Response

Field	Type	Unit	Meaning
`fit.equation`	text	–	The fitted equation, for example `Flow(Opening) = +2458.61*Opening +143.197`.
`fit.terms`	array	–	One entry per power of the opening, highest power first: `power`, `coefficient`, `standard_error`.
`fit.r_squared`	number or null	–	How well the curve fits the points (1 is a perfect fit).
`fit.standard_error_y`	number or null	flow unit	Typical distance between the points and the curve.
`fit.f_statistic`	number or null	–	F statistic of the fit. null when it has no meaning (all flows equal, or the points fit exactly).
`fit.degree_requested`, `degree_used`, `degree_capped`	integer, integer, true/false	–	The degree you asked for, the degree fitted, and whether it was reduced.
`fit.n_points`, `df_residual`	integer	count	Number of points, and points minus the number of coefficients.
`fit.ss_regression`, `ss_residual`	number	(flow unit)²	Regression and residual sums of squares.
`fit.opening_range`	object	%	`min` and `max` opening of your data.
`estimates[]`	array	–	For each requested opening: `opening`, `opening_used`, `flow` (in your flow unit), `extrapolated` (outside the range of your data), `floor_applied`.
`warnings[]`	array of text	–	Plain-language notes, such as that the degree was reduced.

Complete response for the example above (HTTP 200):


```
{
  "status": "success",
  "data": {
    "fit": {
      "degree_requested": 3,
      "degree_used": 1,
      "degree_capped": true,
      "n_points": 3,
      "equation": "Flow(Opening) = +2458.61*Opening +143.197",
      "terms": [
        { "power": 1, "coefficient": 2458.613, "standard_error": 0.9992376 },
        { "power": 0, "coefficient": 143.197, "standard_error": 60.21554 }
      ],
      "r_squared": 0.9999998,
      "standard_error_y": 49.63446,
      "f_statistic": 6054004.0,
      "df_residual": 1,
      "ss_regression": 14914520000.0,
      "ss_residual": 2463.579,
      "opening_range": { "min": 19.7, "max": 89.7 }
    },
    "estimates": [
      {
        "opening": 5.0,
        "opening_used": 19.7,
        "flow": 48577.87,
        "extrapolated": true,
        "floor_applied": false
      },
      {
        "opening": 50.0,
        "opening_used": 50.0,
        "flow": 123073.8,
        "extrapolated": false,
        "floor_applied": false
      },
      {
        "opening": 110.0,
        "opening_used": 89.7,
        "flow": 220680.8,
        "extrapolated": true,
        "floor_applied": false
      }
    ],
    "warnings": [
      "The requested degree 3 was reduced to 1: 3 data points support at most degree 1.",
      "2 of 3 estimate openings lie outside the fitted range 19.7 to 89.7 % and were clamped to the fitted
range."
    ]
  },
  "units": {
    "opening": "%",
    "opening_used": "%",
    "flow": "unit of the supplied flow_values (not converted)",
    "coefficient": "flow unit per (%)^power",
    "standard_error": "flow unit per (%)^power",
    "r_squared": "dimensionless",
    "standard_error_y": "unit of the supplied flow_values",
    "f_statistic": "dimensionless",
    "df_residual": "count",
    "n_points": "count",
    "degree_requested": "count",
    "degree_used": "count",
    "ss_regression": "(flow unit)^2",
    "ss_residual": "(flow unit)^2"
  },
  "meta": { "request_id": "a1b2c3d4e5f6", "elapsed_ms": 0.6 }
}
```


Reading the example

- Three points can only support a straight line, so degree 3 was reduced to 1 (`degree_capped` true) and a warning says so.
- 5 % and 110 % are outside the data (19.7 % to 89.7 %), so they are `extrapolated` and, because `clip_to_fit_range` is true, evaluated at 19.7 % and 89.7 % (`opening_used`).

- The flows in `estimates`, `standard_error_y` and the coefficients are in the unit you used for `flow_values`; the response has no flow unit of its own.

Errors

Error code	HTTP	When it occurs here
`validation_error`	422	A list has fewer than 3 points, the two lists differ in length, the degree is outside 1 to 10, or a value is not a finite number.
`fit_failed`	422	The data cannot be fitted: fewer different opening values than the degree needs. `error.details` gives the numbers.

Example of `fit_failed` (all openings equal), HTTP 422:

```
{
  "status": "error",
  "error": {
    "code": "fit_failed",
    "message": "A degree-2 fit needs at least 3 different opening values; the data contain only 1.",
    "details": { "distinct_opening_values": 1, "required": 3, "degree_used": 2 }
  },
  "request_id": "a1b2c3d4e5f6"
}
```

Notes

- **Units:** you decide the unit of the flow by the numbers you send (kg/h, m³/h, t/d, ...). The response is in that same unit, and the API does not convert anything. Openings are always in %.
- Use the lowest degree that describes your data. A high degree through few points follows the noise and can behave wildly outside the range.
- `null` in a statistic means it is not defined for these data. The API never returns NaN or infinity.
- `meta` holds only `request_id` and `elapsed_ms`, because there is no composition.

SYSTEM

9. Health check

GET /health

http://192.168.11.216:4300/api/health

Confirms that the service is running and tells you which version is deployed. It needs no body and no authentication, and is suitable for uptime monitoring.

Response

Field	Type	Meaning
`status`	text	`ok` while the service is alive.
`version`	text	The deployed API version.

```
{ "status": "ok", "version": "3.0.0" }
```

Errors

None. If the service is down, the request fails at the network level or the web server answers with a 5xx status.